Lab Assignment 5: Binary Search Tree (BST)

Due: July 14, 2025

1. Overview

This assignment requires implementing a Binary Search Tree (BST) as defined in the provided `BinarySearchTree.h` file. The BST optimizes operations such as search, insertion, and traversal by maintaining a sorted order. Your task is to implement the specified methods in `BinarySearchTree.cpp` and submit it for live grading to receive full credit.

2. Implementation Tasks

Implement the following methods for the `BinarySearchTree` class, ensuring compliance with BST properties:

1. **Default Constructor**: Initializes the BST with `root = nullptr`.
2. **Copy Constructor**: Creates a deep copy of another BST using the `recursiveCopy` helper function.
3. `void insert(T newValue)`: Inserts a value of type `T` while preserving BST order, using the `recursiveInsert` method.
4. `Node<T>* search(T aValue) const`: Searches for a value in the BST, leveraging its sorted structure, via the `recursiveSearch` method.
5. **Destructor**: Frees all heap-allocated memory using the `recursiveDestroy` method.
6. `Node<T>* min() const`: Returns the node with the minimum value, found via the `recursiveMin` method.
7. `Node<T>* findParent(const Node<T>* start) const`: Locates the parent of a specified node in the BST.
8. `Node<T>* successor(const Node<T>* start) const`: Finds the in-order successor (the node with the next larger value) using `findParent` and `recursiveRevMin`.
9. `void traverse(std::ostream& out) const`: Performs an in-order traversal, outputting values in ascending order to the provided output stream.
10. `void print(std::ostream& out) const`: Outputs the BST in the following tree-like format:

```
  /--7
5   /--4
  \--3
    \--1
```

11. `void remove(Node<T>* node)`: Removes a specified node, handling the following cases:
    (a) The node is a leaf.
    (b) The node has only a right child (`left = nullptr`).

(c) The node has only a left child (`right = nullptr`).

(d) The node has both children (requires finding the successor via `successor` and recursively calling `remove`).

Note: Handle edge cases, such as attempting to remove a non-existent node. Refer to lectures for additional guidance and examples.

3. Submission Guidelines

Submit the following file via Moodle by **July 14, 2025**:

- `BinarySearchTree.cpp`

Requirements:

- Ensure the file name and extension are exactly as specified.
- Match class names and method signatures to those in `BinarySearchTree.h`.
- Do not compress or archive the file.

Failure to meet these requirements will result in a score of 0 due to automated testing constraints.

4. Support and Clarifications

For assistance, attend lab sessions or post questions on Piazza. Use the "private to instructors" option for any code-related queries to avoid public sharing, as public code sharing is prohibited.